The Agile System Development Lifecycle (SDLC)

The goal of this article is to describe the agile system development lifecycle (SDLC), the agile SDLC, putting it in context from what you may have heard about within the agile community and more importantly within the context of your overall IT efforts.

This article covers:

1. The scope of lifecycles (https://ambysoft.com/essays/agileLifecycle.html#LifecycleScope)
2. Concept: Pre-planning (https://ambysoft.com/essays/agileLifecycle.html#ConceptPhase)
3. Inception (https://ambysoft.com/essays/agileLifecycle.html#Ideation)
4. Construction sprints (https://ambysoft.com/essays/agileLifecycle.html#Development)
5. Release sprints (https://ambysoft.com/essays/agileLifecycle.html#Release)
6. Production (https://ambysoft.com/essays/agileLifecycle.html#Production)
7. Retirement (https://ambysoft.com/essays/agileLifecycle.html#Retirement)

1. The Scope of Lifecycles

Although some lifecycles are depicted otherwise, the Scrum construction lifeycle of Figure 1 (https://ambysoft.com/essays/agileLifecycle.html#FigureScrum) is a classic example of such, the reality on the ground is much different. Figure 2 (https://ambysoft.com/essays/agileLifecycle.html#FigureAgileProject) depicts an agile, Scrum-based project lifecycle, adding an Ideation/Sprint 0 phase and a Deploy phase. Figure 3 (https://ambysoft.com/essays/agileLifecycle.html#FigureDetailedLifecycle) goes one step further to

present a full, agile solution lifecycle. Each of these lifecycles are described in The Scrum Lifecycles: Construction, Project, and Solution (https://ambysoft.com/essays/agileLifecycleProject.html)

Figure 1. The Scrum construction lifecycle (click to enlarge).

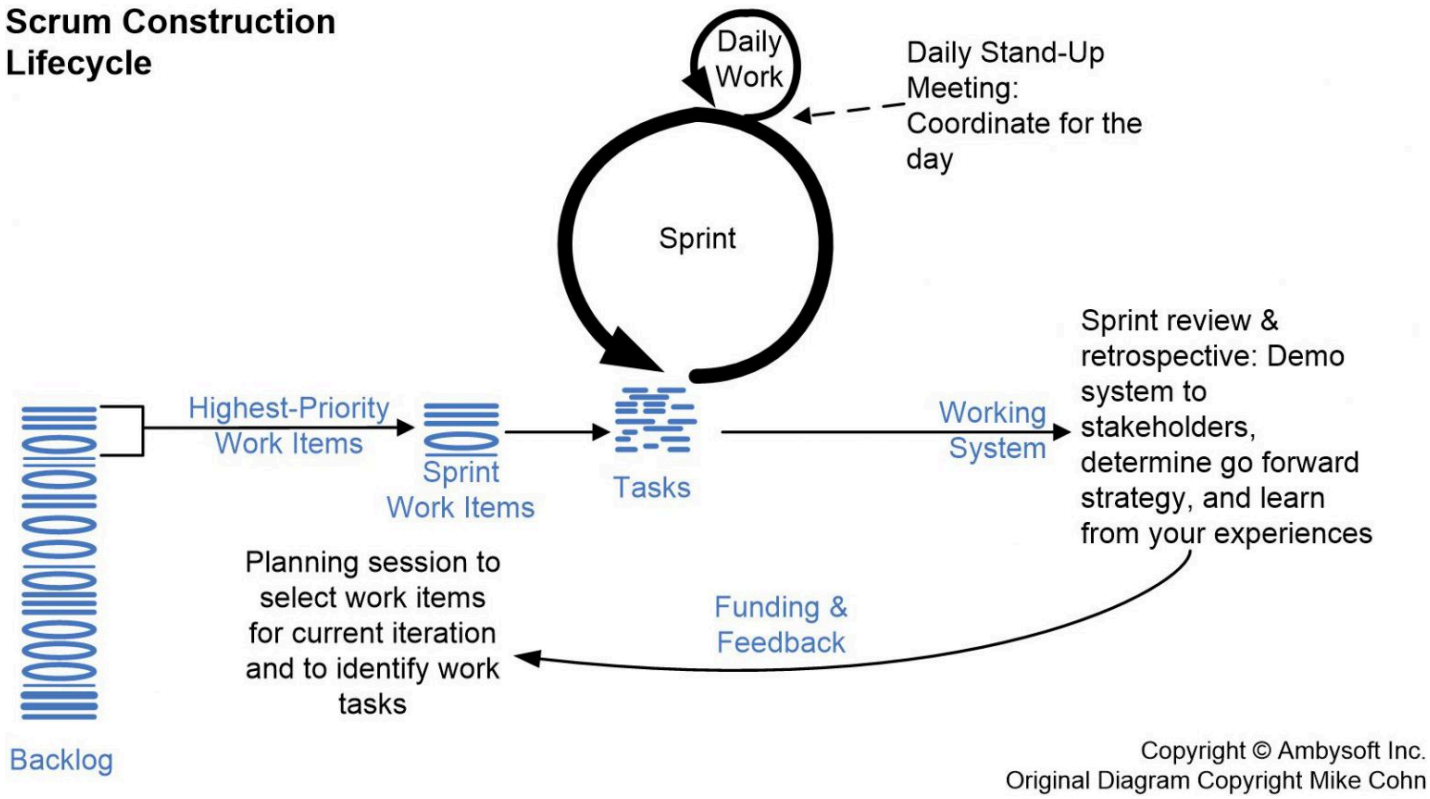


Figure 2. The agile project lifecycle (click to enlarge).

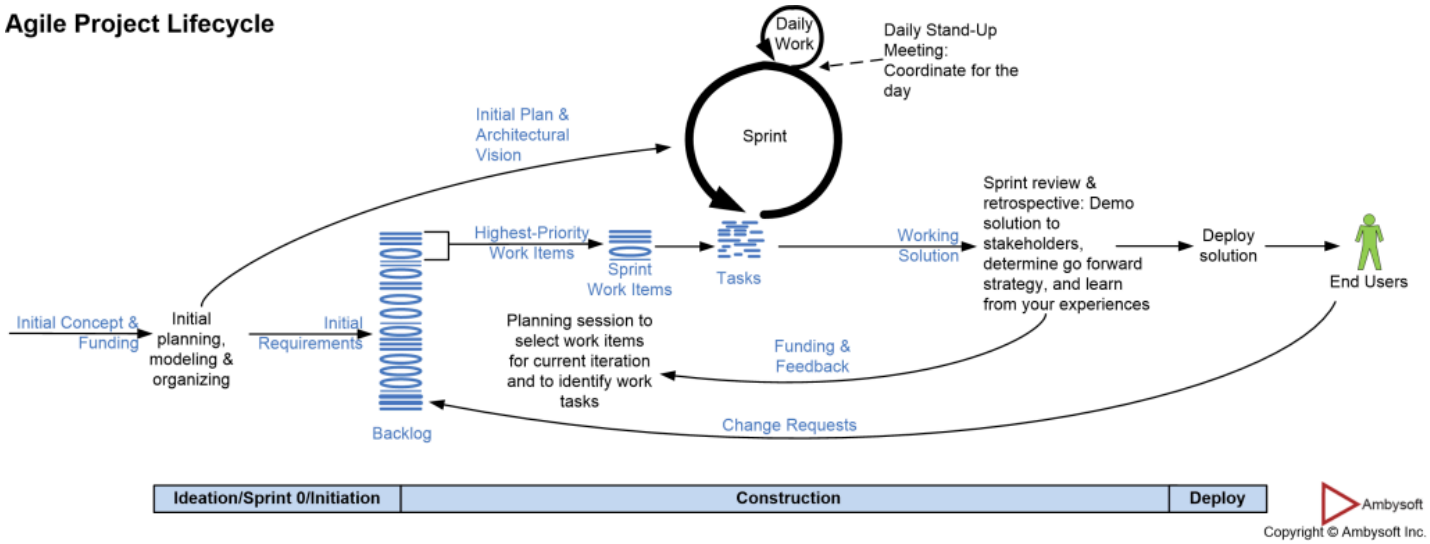


Figure 3. A detailed agile SDLC (click to enlarge).

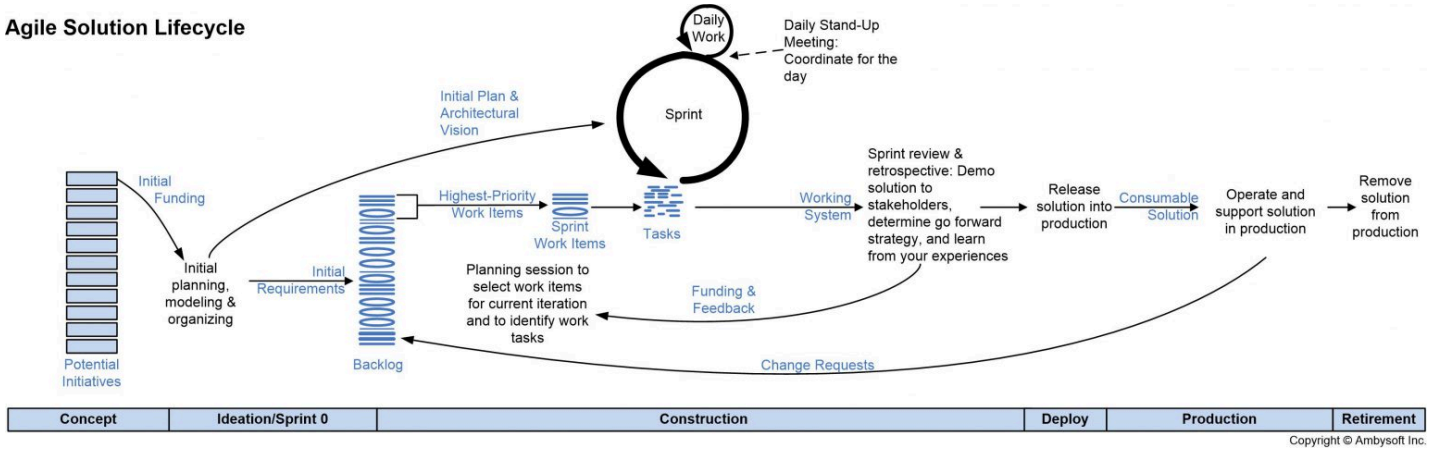
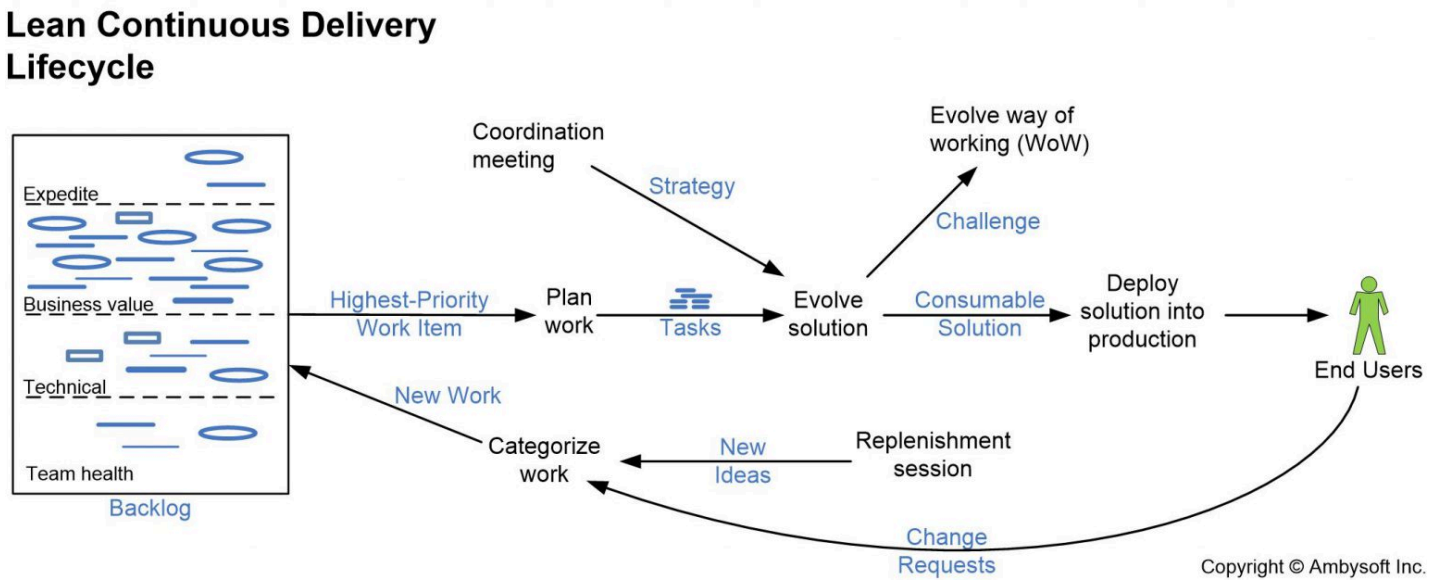


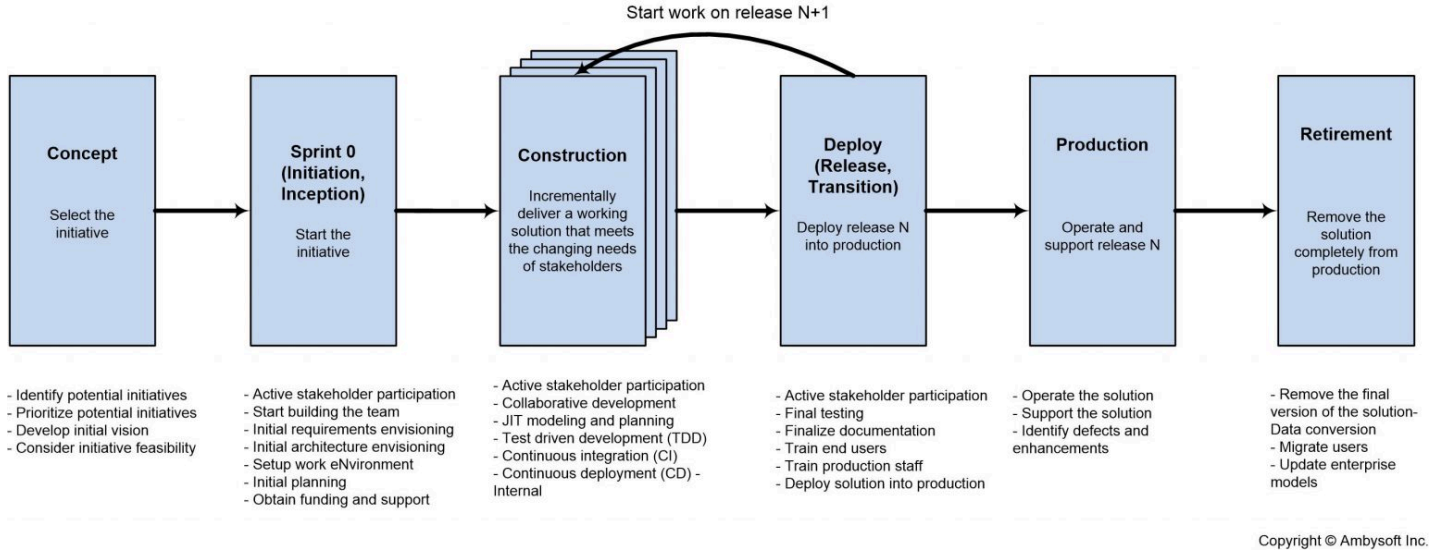
Figure 4 (https://ambysoft.com/essays/agileLifecycle.html#FigureLeanCD) depicts a lean, Kanban-based continuous delivery lifecycle that is well suited for a DevOps approach. This lifecycle is described in greater detail in The Continuous Delivery (CD) Software Lifecycle (https://ambysoft.com/essays/agileLifecycleContinuousDelivery.html).

Figure 4. The lean continuous delivery (click to enlarge).



On the surface, the agile SDLC of Figure 5 (https://ambysoft.com/essays/agileLifecycle.html#FigureHighLevelLifecycle) looks very much like a traditional SDLC, but when you dive deeper you quickly discover that this isn't the case. This is particularly true when you consider the detailed view of Figure 3 (https://ambysoft.com/essays/agileLifecycle.html#FigureDetailedLifecycle). Because the agile SDLC is highly collaborative, iterative, and incremental the roles which people take are much more robust than on traditional initiatives (https://scottambler.com/initiative/). In the traditional world a business analyst (https://agilemodeling.com/essays/businessAnalysts.htm) created a requirements model (https://agilemodeling.com/essays/agileRequirements.htm) that is handed off to an architect (https://agilemodeling.com/essays/agileArchitecture.htm) who creates design (https://agilemodeling.com/essays/agileDesign.htm) models that are handed off to a coder who writes programs which are handed off to a tester (https://ambysoft.com/essays/floot.html) and so on. On an agile team, developers work closely with their stakeholders to understand their needs, they pair together to implement and test their solution, and the solution is shown to the stakeholder for quick feedback. Instead of specialists handing artifacts to one another, and thereby injecting defects at every step along the way, agile developers are generalizing specialists (https://agilemodeling.com/essays/generalizingSpecialists.htm) with full lifecycle skills.

Figure 5. The Agile SDLC (high-level). Click to enlarge.



(https://Ambysoft.com/wp-content/uploads/2023/03/agileLifecycle-scaled.jpg)

2. The Concept Phase: Portfolio Planning

The Concept Phase, sometimes called sprint -1, is the pre-initiative (https://scottambler.com/initiative/) aspects of portfolio management (http://www.enterpriseunifiedprocess.com/essays/portfolioManagement.html). During this phase you will:

1. **Define the business opportunity**. You must consider the bigger business picture and focus on market concerns. This includes exploring how the new functionality will improve your organization's presence in the market, how it will impact profitability, and how it will impact the people within your organization. This exploration effort should be brief, not all initiatives (https://scottambler.com/initiative/) will make the initial cut so you only want to invest enough effort at this point to get a good gut feel for the business potential.
2. **Identify a viable strategy for the initiative**. There are several issues to consider when identifying a potential strategy for the initiative. For example, do you build a new system or buy an existing package and modify it? If you decide to build, do you do so onshore or offshore? Will the work be solely done by your own development team, by a team from a system integrator (SI), or in partnership with the SI? What development paradigm – traditional/waterfall, iterative, or agile – will you follow? Will the team be co-located, near-located within the same geographic region, or far-located around the world? As you can see there are many combinations of strategy available to you, and at this point in time you may only be able to narrow the range of the possibilities but be forced to leave the final decision to the team in future sprints.

3. **Assess the feasibility**. During the Concept Phase you will want to do just enough feasibility analysis to determine if it makes sense to invest in the potential initiative (https://scottambler.com/initiative/). Depending on the situation you may choose to invest very little effort in considering feasibility, for many systems just considering these issues for a few minutes is sufficient for now, and for some systems you may choose to invest days if not weeks exploring feasibility. Many organizations choose to do just a little bit of feasibility analysis during the Concept Phase, and then if they decide to fund the effort they will invest more effort during Inception (https://ambysoft.com/essays/agileLifecycle.html#Ideation). In my experience you need to consider four issues when exploring feasibility (https://ambysoft.com/essays/projectjustification.html): economic feasibility, technical feasibility, operational feasibility, and political feasibility. Your feasibility analysis efforts should also produce a list of potential risks and criteria against which to make go/no-go decisions at key milestone points during your initiative. You should question the feasibility of the initiative (https://scottambler.com/initiative/) throughout the lifecycle to reduce overall risk.

Concept Phase activities can and should be as agile as you can possibly make it – you should collaborate with stakeholders who are knowledgeable enough and motivated enough to consider this potential initiative and invest in just enough effort to decide whether to consider funding the effort further.

3. Ideation/Sprint 0: Getting Your Initiative Going

The first week or so of an agile initiative is often referred to as Sprint 0 or Ideation. Your goal during this period is to start the initiative (https://scottambler.com/initiative/) by:

1. **Garnering initial support and funding for the initiative**. This may have been already achieved via your portfolio management efforts, but realistically at some point somebody is going to ask what are we going to get, how much is it going to cost, and how long is it going to take. You need to be able to provide reasonable, although potentially evolving, answers to these questions if you're going to get permission to work on the effort. You may need to justify your initiative via a feasibility study (https://ambysoft.com/essays/projectjustification.html).
2. **Actively working with stakeholders to** initially model **the scope of the system**. As you see in Figure 6 (https://ambysoft.com/essays/agileLifecycle.html#FigureAMDD), during sprint 0 agilists will do some initial requirements modeling (https://agilemodeling.com/essays/initialRequirementsModeling.htm) with their stakeholders to

identify the initial, albeit high-level, requirements for the system. To promote active stakeholder participation (https://agilemodeling.com/essays/activeStakeholderParticipation.htm) you should use inclusive tools (https://agilemodeling.com/essays/inclusiveModels.htm), such as index cards and white boards to do this modeling – our goal is to understand the problem and solution domain, not to create mounds of documentation. The details of these requirements are modeled on a just in time (JIT) basis in model storming (https://agilemodeling.com/essays/amdd.htm#ModelStorming) sessions during the development cycles (https://ambysoft.com/essays/agileLifecycle.html#Development).

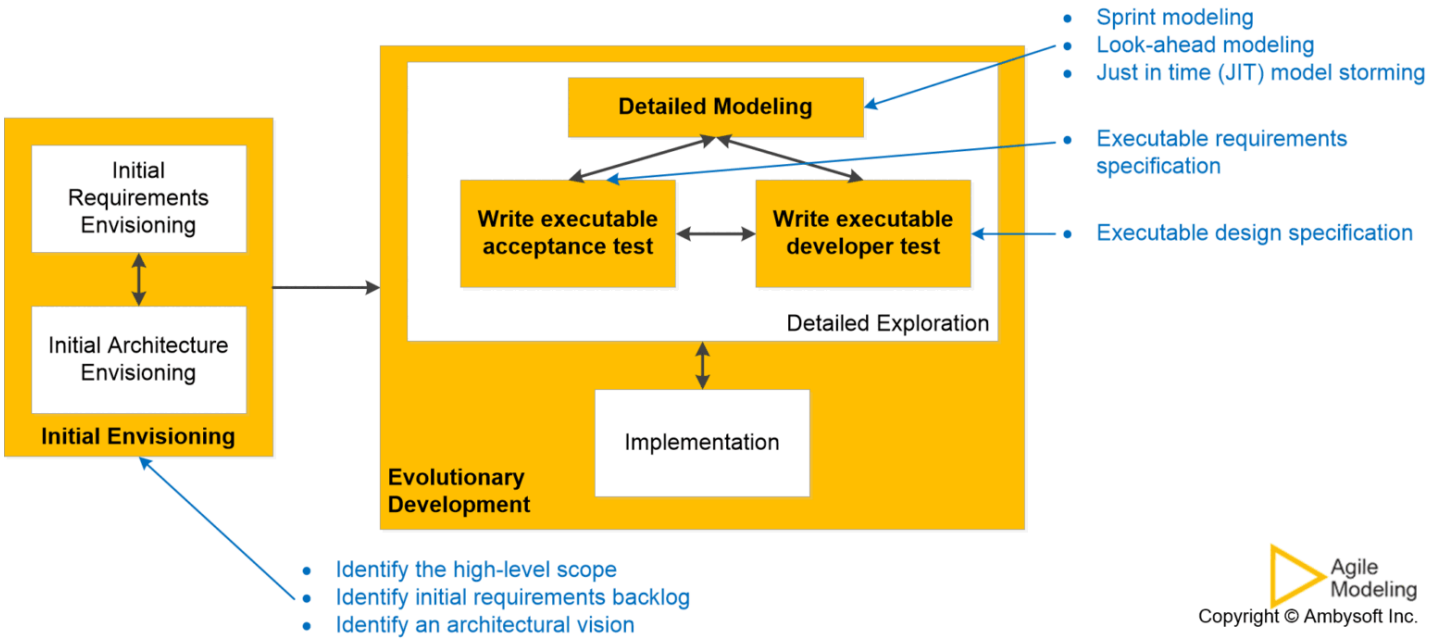
3. **Starting to build the team**. Although your team will evolve over time, at the beginning of a development effort you will need to start identifying key team members and start bringing them onto the team. At this point you will want to have at least one or two senior developers, the team leader, and one or more stakeholder representatives.

4. **Modeling an** initial architecture (https://agilemodeling.com/essays/initialArchitectureModeling.htm) **for the system**. Early in the project you need to have at least a general idea of how you're going to build the system. Is it a mainframe COBOL application? A .Net application? J2EE? Something else? As you see in Figure 6 (https://ambysoft.com/essays/agileLifecycle.html#FigureAMDD), the developers on the team will get together in a room, often around a whiteboard, discuss and then sketch out a potential architecture for the system. This architecture will likely evolve over time, it will not be very detailed yet (it just needs to be good enough (https://agilemodeling.com/essays/barelygoodenough.htm) for now), and very little documentation (if any) needs to be written. The goal is to identify an architectural strategy, not write mounds of documentation. You will work through the design details later during development cycles (https://ambysoft.com/essays/agileLifecycle.html#Development) in model storming (https://agilemodeling.com/essays/amdd.htm#ModelStorming) sessions and via TDD (https://agiledata.org/essays/tdd.html).

5. **Setting up the environment**. You need workstations, development tools, a work area, … for the team. You don't need access to all of these resources right away, although at the start of the initiative you will need most of them.

6. **Estimating the effort**. You'll need to put together an initial estimate for your agile initiative (http://www.drdobbs.com/architect/223100694?cid=Ambysoft) based on the initial requirements, the initial architecture, and the skills of your team. This estimate will evolve throughout the effort.

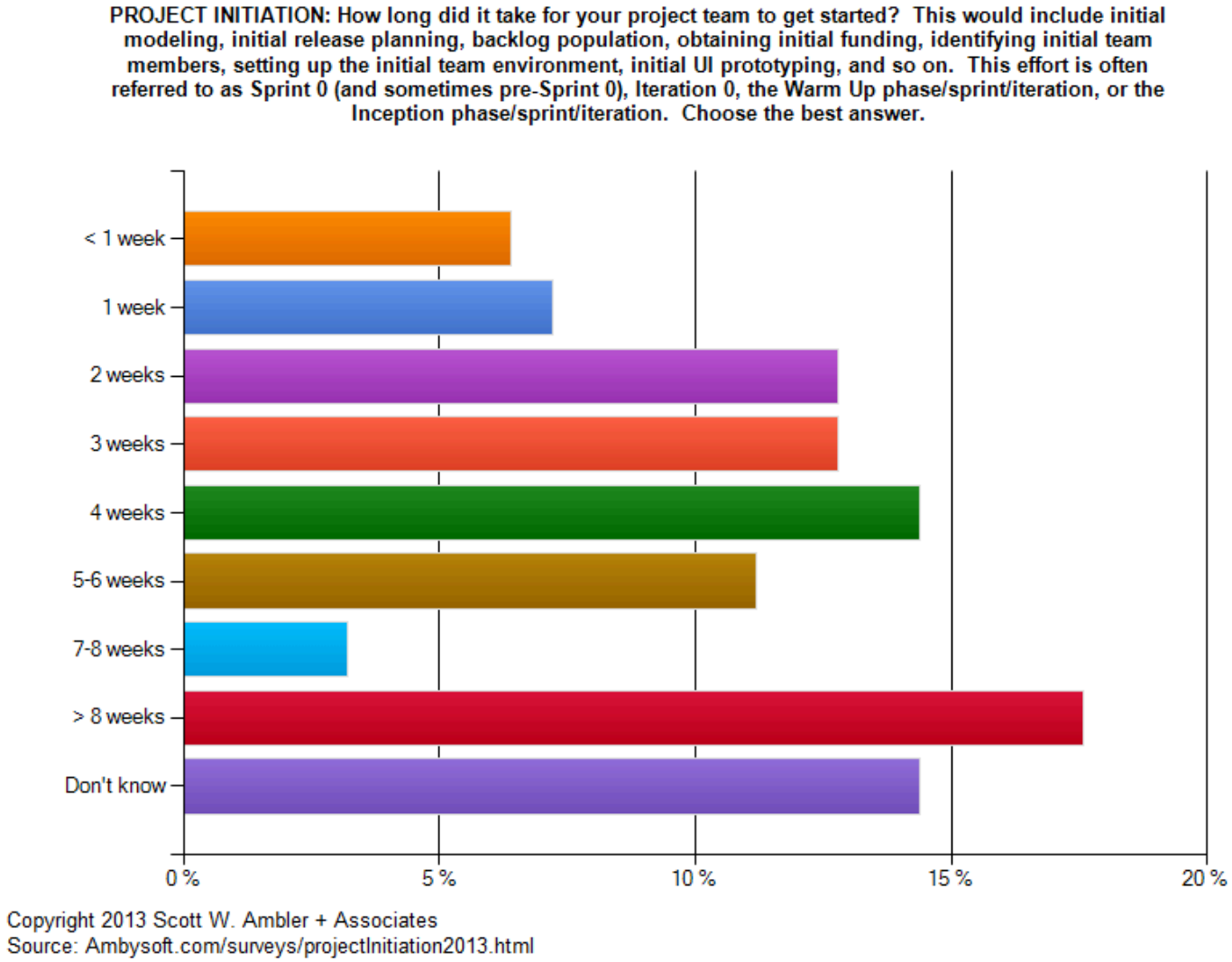
Figure 6: The Agile Model Driven Development (AMDD) lifecycle (click to enlarge).



(https://Ambysoft.com/wp-content/uploads/2023/03/AMDD.gif)

The 2013 Agile Project Initiation Survey (https://ambysoft.com/surveys/projectInitiation2013.html) found that the average time to initiate an agile project took 4.6 weeks. Figure 7 (https://ambysoft.com/essays/agileLifecycle.html#FigureInceptionLength) depicts the range of initiation periods. Differences are the results of the complexity of the domain/problem space, technical complexity of what you're trying to accomplish, availability of stakeholders, ability of stakeholders to come to agreement as to the scope. and ability of the team to form itself and to obtain necessary resources.

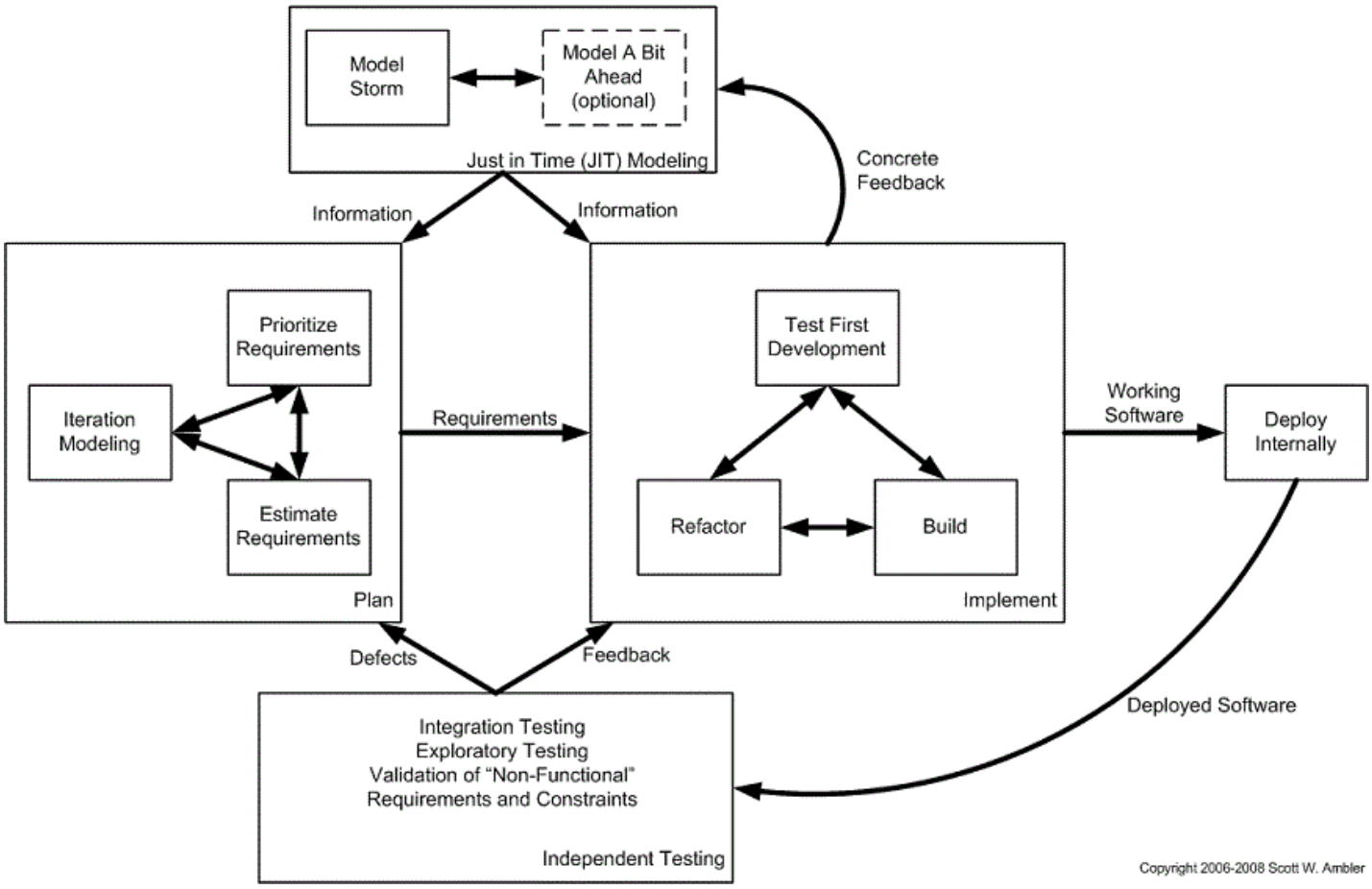
Figure 7. How long did it take to initiate an agile project?



4. Construction Sprints

During construction sprints agilists incrementally deliver high-quality working software which meets the changing needs of our stakeholders, as overviewed in Figure 8 (https://ambysoft.com/essays/agileLifecycle.html#FigureDevelopmentCycle).

Figure 8. Agile software development process during a construction sprint.



(https://Ambysoft.com/wp-content/uploads/2023/03/agileDevelopmentCycle.gif)

We achieve this by:

1. **Collaborating closely with both our stakeholders and with other developers**. We do this to reduce risk through tightening the feedback cycle (https://ambysoft.com/essays/whyAgileWorksFeedback.html) and by improving communication via closer collaboration (https://agilemodeling.com/essays/communication.htm).
2. **Implementing functionality in priority order**. We allow our stakeholders to change the requirements (https://agilemodeling.com/essays/changeManagement.htm) to meet their exact needs as they see fit. The stakeholders are given complete control over the scope, budget, and schedule (https://ambysoft.com/essays/brokenTriangle.html) – they get what they want and spend as much as they want for as long as they're willing to do so.
3. **Analyzing and designing.** We analyze individual requirements by model storming (https://agilemodeling.com/essays/amdd.htm#ModelStorming) on a just-in-time (JIT) basis for a few minutes before spending several hours or days implementing the requirement. Guided by

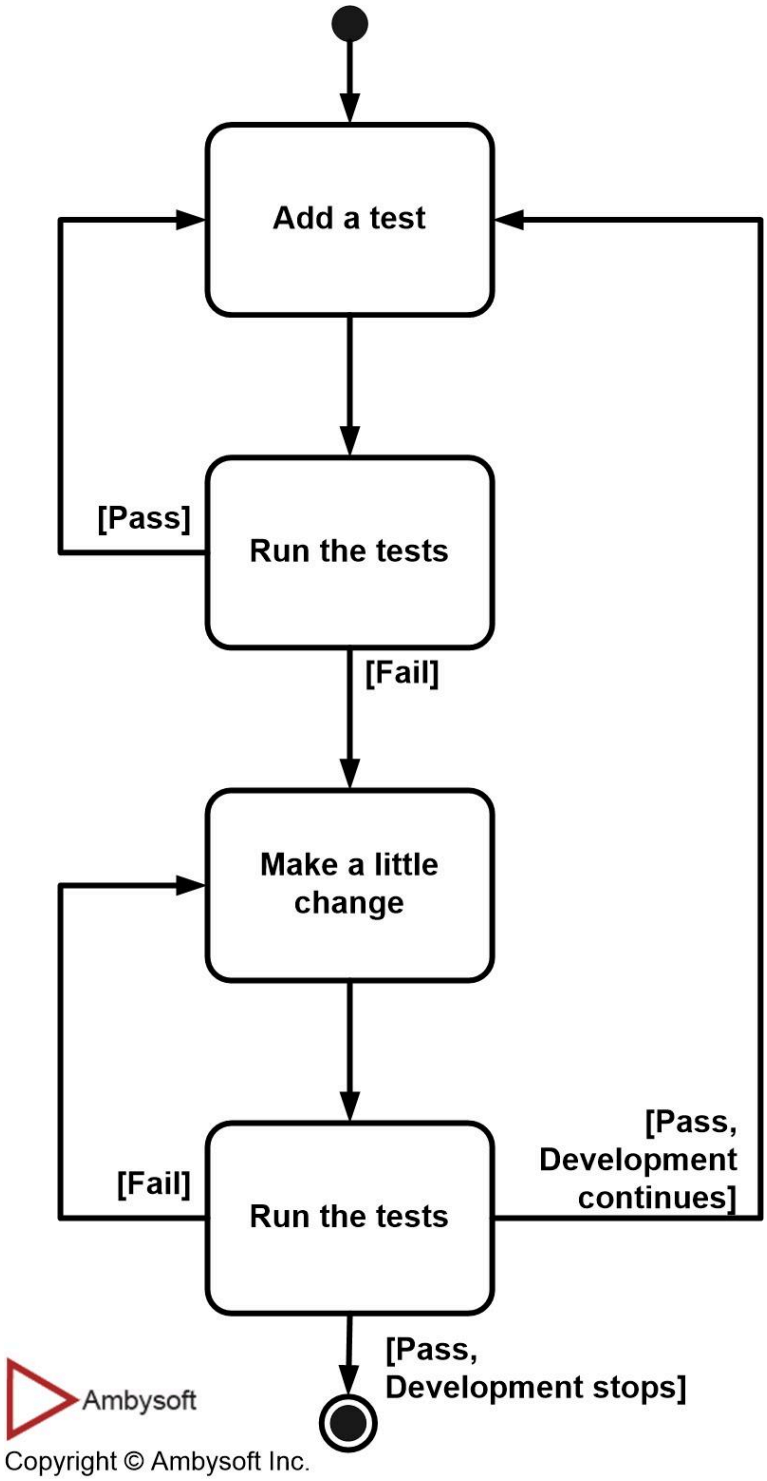
our architecture models, often hand-sketched diagrams, we take a highly-collaborative, test-driven design (TDD) (https://agiledata.org/essays/tdd.html) approach to development (see Figure 9 (https://ambysoft.com/essays/agileLifecycle.html#FigureTestFirst)) where we iteratively write a test and then write just enough production code to fulfill that test. Sometimes, particularly for complex requirements or for design issues requiring significant forethought, we will model just a bit ahead (https://agilemodeling.com/essays/modelAhead.htm) to ensure that the developers don't need to wait for information.

4. **Ensuring quality**. Disciplined agilists are firm believers in following guidance such as coding conventions (https://ambysoft.com/essays/codingGuidelines.html) and modeling style guidelines (https://agilemodeling.com/style/). Furthermore, we refactor (https://agiledata.org/essays/databaseRefactoring.html#Refactoring) our application code and/or our database schema (https://agiledata.org/essays/databaseRefactoring.html#DatabaseRefactoring) as required to ensure that we have the best design possible.

5. **Regularly delivering working solutions**. At the end of each development cycle/sprint you should have a partial, working solution to show people. Better yet, you should be able to deploy this solution into a pre-production testing/QA sandbox for system integration testing. The sooner, and more often, you can do such testing the better. See Agile Testing and Quality Strategies: Discipline Over Rhetoric (https://ambysoft.com/essays/agileTesting.html) for more thoughts.

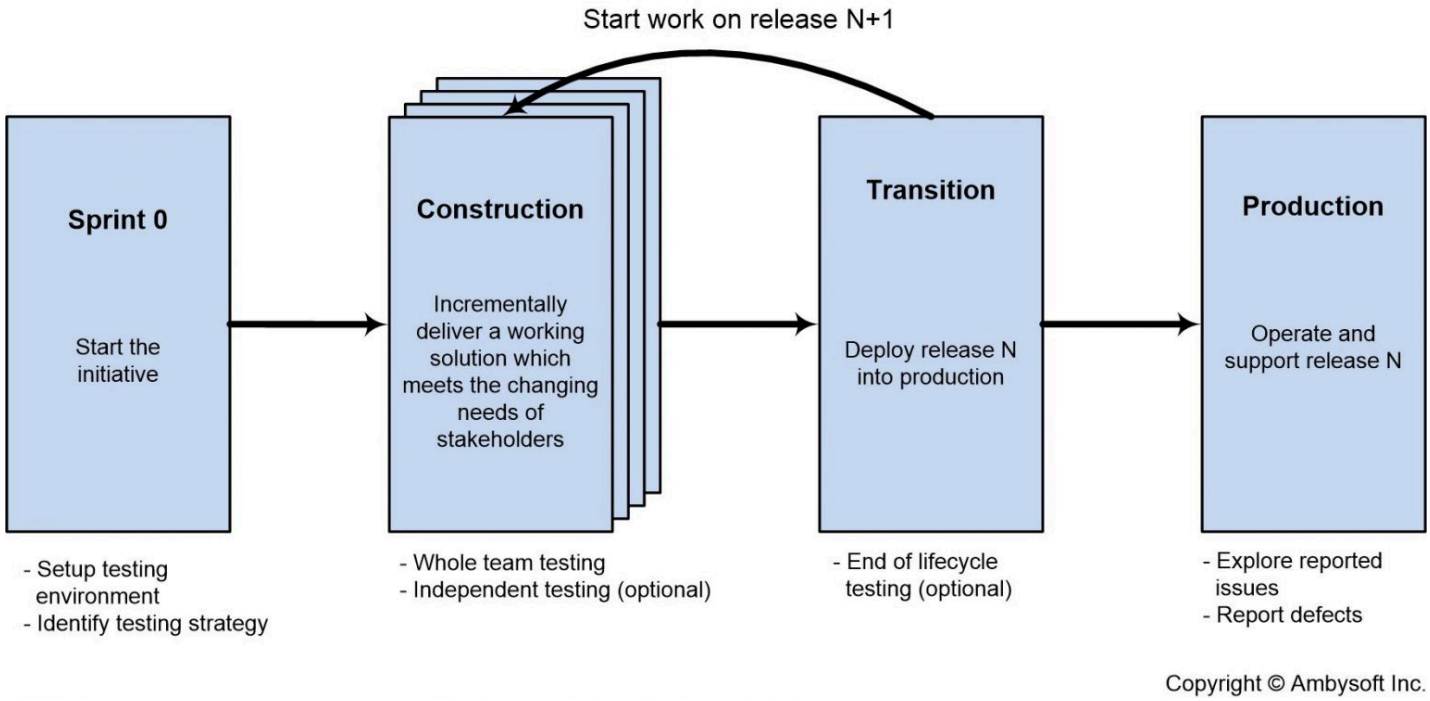
6. **Testing, testing, and yes, testing**. As you can see in Figure 10 (https://ambysoft.com/essays/agileLifecycle.html#FigureTestingDuringDevelopment) agilists do a significant amount of testing (https://ambysoft.com/essays/agileTesting.html) throughout construction. As part of construction we do confirmatory testing, a combination of developer testing at the design level and agile acceptance testing at the requirements level. In many ways confirmatory testing is the agile equivalent of "testing against the specification" because it confirms that the software which we've built to date works according to the intent of our stakeholders as we understand it today. This isn't the complete testing picture: Because we are producing working software on a regular basis, at least at the end of each sprint although ideally more often, we're in a position to deliver that working software to an independent test team for investigative testing. Investigative testing is done by test professionals who are good at finding defects which the developers have missed. These defects might pertain to usability or integration problems, sometimes they pertain to requirements which we missed or simply haven't implemented yet, and sometimes they pertain to things we simply didn't think to test for.

Figure 9. Taking a “test first” approach to construction.



(https://Ambysoft.com/wp-content/uploads/2023/03/tddSteps.jpg)

Figure 10. Testing activities during an agile initiative (click to enlarge).



(https://Ambysoft.com/wp-content/uploads/2023/03/agileLifecycleTestingSDLC.jpg)

> I would rather fail three weeks into a six-month initiative than ten months into a six-month initiative.

5. Deploy: The “End Game”

During the deploy phase, also known as the “end game” or transition, we release the solution into production (https://ambysoft.com/essays/agileLifecycle.html#Production). Not that for complex systems the end game may prove to be several sprints, although if you’ve done system and user testing during construction sprints (as indicated by Figure 7 (https://ambysoft.com/essays/agileLifecycle.html#FigureDevelopmentCycle)) this likely won’t be the case. There are several important aspects to this effort:

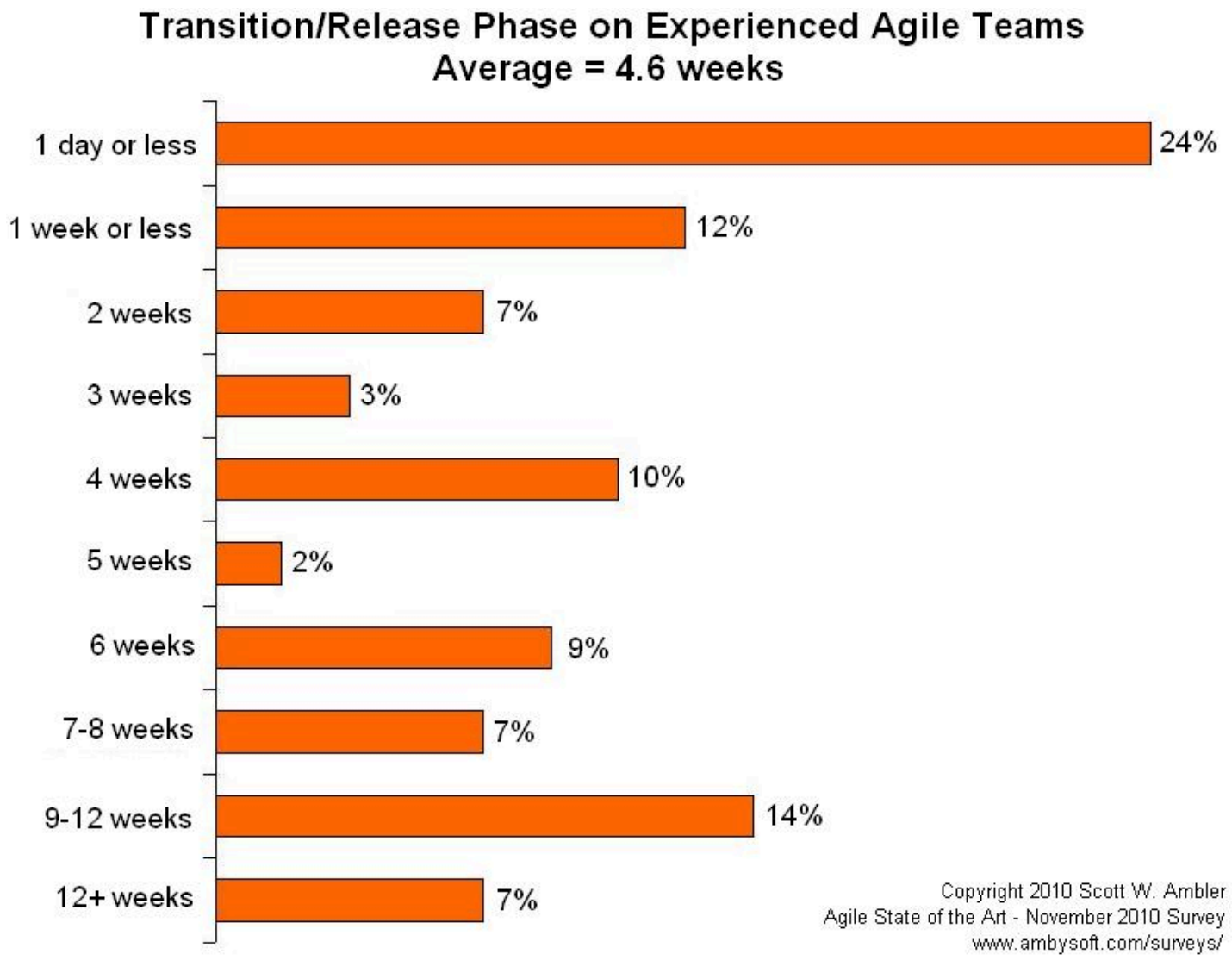
1. **Final testing of the system**. Final system and acceptance testing should be performed at this point, although as I pointed out earlier the majority of testing should be done during

construction sprints (ideally, you just need to rerun your regression test suite to see that it works). You may choose to pilot/beta test your system with a subset of the eventual end users. See the article Agile Testing and Quality Strategies: Discipline Over Rhetoric (https://ambysoft.com/essays/agileTesting.html) for more thoughts on testing.

2. **Rework**. There is no value testing the system if you don't plan to act on the defects that you find. You may not address all defects, but you should expect to fix some of them.
3. **Finalization of any system and user documentation**. Some documentation (https://agilemodeling.com/essays/agileDocumentation.htm) may have been written during construction sprints, but it typically isn't finalized until the system release itself has been finalized to avoid unnecessary rework Note that documentation is treated like any other requirement: it should be costed, prioritized, and created only if stakeholders are willing to invest in it. Agilists believe that if stakeholders are smart enough to earn the money then they must also be smart enough to spend it appropriately.
4. **Training**. We train end users, operations staff, and support staff to work effectively with our system.
5. **Deploy the system**. See my article entitled System Deployment Tips and Techniques (https://ambysoft.com/essays/deploymentTips.html).

As you can see in Figure 11 (https://ambysoft.com/essays/agileLifecycle.html#FigureTransitionLength), on average agile teams take 4.6 weeks to transition their system into production according to the November 2010 Agile State of the Art Survey (https://ambysoft.com/surveys/agileStateOfArt201011.html). As you can see in the figure there is a wide range of time taken. I believe this variance ranges based on the complexity of the solution, the amount of deployment automation (teams that have adopted a continuous deployment strategy automate many of the technical aspects of transition), the comprehensiveness of the agile testing (https://ambysoft.com/essays/agileTesting.html) effort during construction, the need for manual efforts such as training and educating end users (or support or operations staff), and the organizational complexity of your environment.

Figure 11. Amount of time experienced agile teams invested in releasing/transitioning their solution into production.



6. Production

The goal of the Production Phase (http://www.enterpriseunifiedprocess.com/essays/productionPhase.html) is to keep systems useful and productive after they have been deployed to the user community. This process will differ from organization to organization and perhaps even from system to system, but the fundamental goal remains the same: keep the system running and help users to use it. Shrink-wrapped software, for example, will not require operational support but will typically require a help desk to assist users. Organizations that implement systems for internal use will usually require an operational staff to run and monitor systems.

This phase ends when the release of a system has been slated for retirement or when support for that release has ended. The latter may occur immediately upon the release of a newer version, some time after the release of a newer version, or simply on a date that the business has decided

to end support. This phase typically has one sprint because it applies to the operational lifetime of a single release of your software. There may be multiple sprints, however, if you defined multiple levels of support that your software will have over time.

7. Retirement

The goal of the Retirement Phase (http://www.enterpriseunifiedprocess.com/essays/retirementPhase.html) is the removal of a system release from production, and occasionally even the complete system itself, an activity also known as system decommissioning or system sunsetting. Retirement of systems is a serious issue faced by many organizations today as legacy systems are removed and replaced by new systems. You must strive to complete this effort with minimal impact to business operations. If you have tried this in the past, you know how complex it can be to execute successfully. System releases are removed from production for several reasons, including:

1. **The system is being complete replaced**. It is not uncommon to see homegrown systems for human resource functions being replaced by COTS systems such as SAP or Oracle Financials.
2. **The release is no longer to be supported**. Sometimes organizations will have several releases in production at the same time, and over time older releases are dropped.
3. **The system no longer needed to support the current business model**. A organization may explore a new business area by developing new systems only to discover that it is not cost effective.
4. **The system is redundant**. Organizations that grow by mergers and/or acquisitions often end up with redundant systems as they consolidate their operations.
5. **The system has become obsolete**.

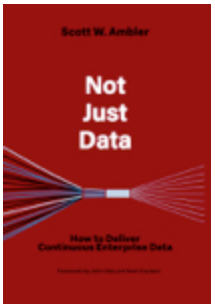
In most cases, the retirement of older releases is a handled during the deployment of a newer version of the system and is a relatively simple exercise. Typically, the deployment of the new release includes steps to remove the previous release. There are times, however, when you do not retire a release simply because you deploy a newer version. This may happen if you can not require users to migrate to the new release or if you must maintain an older system for backward compatibility.

Related Resources

- The Continuous Delivery (CD) Software Lifecycle (https://ambysoft.com/essays/agileLifecycleContinuousDelivery.html)
- Do Agile Lifecyles Have Phases? Yes! (https://ambysoft.com/essays/agileLifecyclePhases.html)
- The Scrum Lifecycles: Construction, Project, and Solution (https://ambysoft.com/essays/agileLifecycleProject.html)

About the Author

Scott W. Ambler (https://ScottAmbler.com) is a data methodologist who works with organizations to help them improve their ways of working (WoW). Scott is an international keynote speaker, a trainer, the (co-)author of 31 books (https://scottambler.com/my-books/), the co-creator of the Disciplined Agile (DA) toolkit, and a former Vice President at PMI. Scott has earned two Master degrees, one in artificial intelligence (AI) and another in information science.



(https://scottambler.com/not-just-data-how-to-deliver-continuous-enterprise-data/)

Scott's latest book is Not Just Data: How to Deliver Continuous Enterprise Data (https://scottambler.com/not-just-data-how-to-deliver-continuous-enterprise-data/), published in February 2026. Every organization wants an enterprise data ecosystem that enables better-informed, data-driven decision-making. They also want to operationalize effective artificial intelligence (AI) to streamline their workflow, better serve customers, and improve the bottom line. To accomplish these goals, they require a continuous flow of enterprise data.

This book is written as a management novel, along the lines of *The Goal* and *The Phoenix Project*. The story follows the journey of an enterprise data warehousing team at a Fortune 500 company, sharing lessons and insights from decades of hard-earned experience. The story and its lessons are pragmatic and useful for both technical and non-technical readers.

(https://scottambler.com/)

Ambysoft

(https://ambysoft.com/)

(https://agilemodeling.com/)

(https://agiledata.org/)

(https://twitter.com/scottambler) (https://www.youtube.com/channel/UCItFj5rq20L_c9y458cOww)

Copyright (https://scottambler.com/copyright/) 1996-2026 Ambysoft Inc.
(https://ambysoft.com/)

This site is owned by Ambysoft Inc. (https://ambysoft.com/)